

DREAM: Dynamic Resilient Spatio-Semantic Memory with Hybrid Localization for Mobile Manipulation

Zhijie Yan, Shufei Li, Ze Zhang, Xin Liu, Yuhang Zheng, Zuoxu Wang

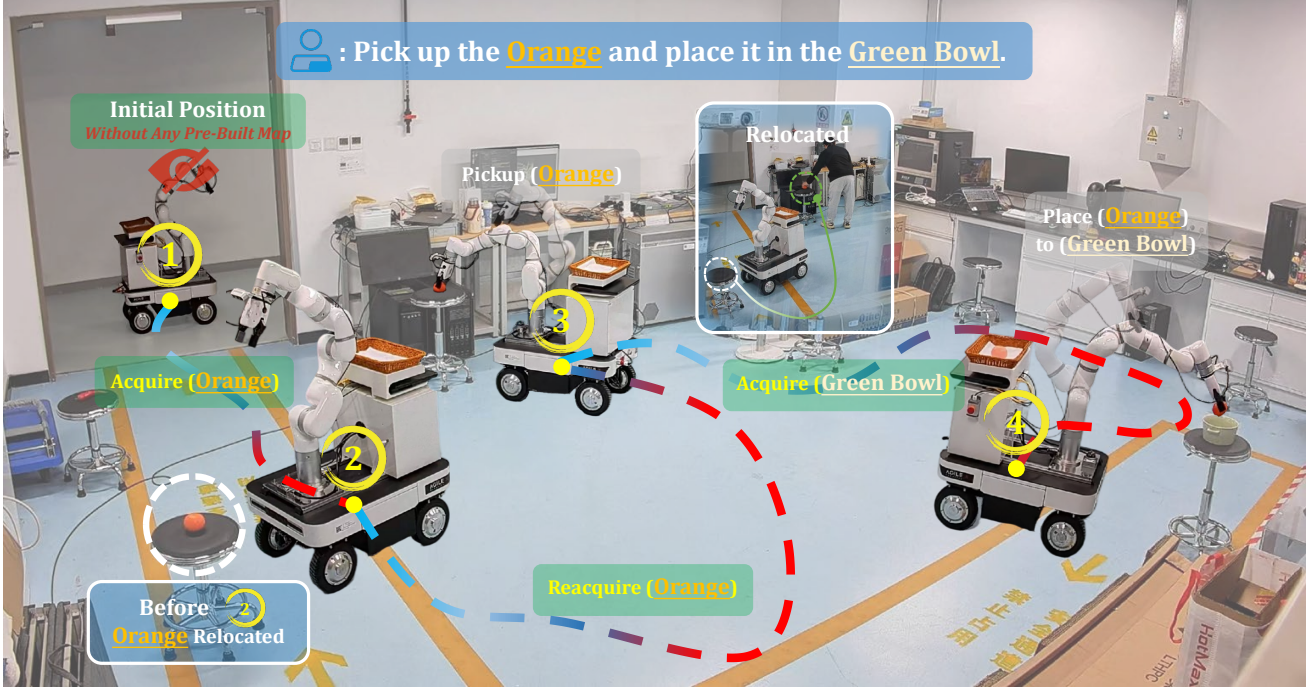


Fig. 1. **Overview of the DREAM framework.** DREAM is a mobile manipulation framework for dynamic, previously unseen indoor environments without any pre-built map. Given a natural-language instruction, the robot actively explores the scene to acquire the target object. When human intervention relocates the object, DREAM leverages hybrid localization and a dynamic spatio-semantic memory to reacquire the target, continuously updating task-relevant scene entities and replanning its actions. This enables consistent scene understanding and reliable mobile manipulation under frequent environment changes.

Abstract—Reliable mobile manipulation in dynamic indoor environments requires a 3D semantic representation that remains consistent with the evolving real world. However, most existing systems rely on pre-built maps, assume static environments, or presuppose highly accurate camera poses. When these assumptions break in practice, the internal scene representation quickly drifts from reality, causing navigation and manipulation to operate on stale information and degrading overall performance. We present DREAM, a mobile manipulation framework that tightly couples perception, memory, localization, navigation, and manipulation for previously unseen indoor environments without any pre-built map. DREAM integrates a lightweight indoor LiDARInertialVisual SLAM backend with a dynamic spatio-semantic memory that updates as the scene changes and employs a Redundancy-Aware Memory Pruning (RMP) strategy to discard stale, redundant information, improving robustness in dynamic settings. On top of this representation, a hybrid localization and task-oriented navigation module acquires and reacquires targets after relocations through active exploration. We deploy DREAM on a real mobile manipulation robot in four dynamic indoor laboratory environments. Experiments on navigation, pickup, placement, and long-horizon mobile manipulation

show that DREAM maintains more consistent scene representations and achieves higher task success and reliability than a strong voxel-map baseline, while using substantially less spatio-semantic memory and computation.

Index Terms—Spatio-Semantic Memory, Dynamic Environments, Hybrid Localization, Mobile Manipulation.

I. INTRODUCTION

MOBILE MANIPULATION in real-world settings must operate as a closed loop spanning perception, spatio-semantic memory, localization, navigation, and manipulation in order to cope with dynamic and unpredictable environment changes. Human activities, other agents, and the robot itself continuously modify object layouts, occlusions, and accessibility, so the scene state is inherently time-varying. In such conditions, relying on pre-built maps [1], [2], static-environment assumptions [3], or presupposing highly accurate, drift-free camera poses [4], [5] quickly makes the internal scene representation inconsistent with the physical world. When obstacles are added or removed, task-relevant objects are relocated outside the robot’s field of view, or accumulated drift and tracking failures corrupt the estimated camera trajectory, the robot continues to navigate and manipulate based on obsolete or inaccurate maps, leading to brittle behavior and degraded performance in dynamic settings.

Corresponding author: Zuoxu Wang, e-mail: zuoxu_wang@buaa.edu.cn.

¹Zhijie Yan, Zuoxu Wang, Ze Zhang, Xin Liu and Jihong Liu are with School of Mechanical Engineering and Automation, Beihang University, Beijing 100191, China.

²Shufei Li is with Department of Systems Engineering, City University of Hong Kong, Hong Kong SAR 518057, China.

³Yuhang Zheng is with School of Computing, National University of Singapore, 21 Lower Kent Ridge Rd 119077, Singapore.

In this work, we present DREAM, a mobile manipulation framework for dynamic, previously unseen indoor environments without any pre-built map. Centered on a dynamic spatio-semantic memory, the framework tightly integrates perception, localization, navigation, and manipulation, enabling the robot to maintain a 3D semantic representation that remains aligned with the evolving scene throughout task execution. DREAM employs a lightweight indoor LiDAR-Inertial-Visual multi-sensor SLAM backend, where LiDAR-Inertial odometry [6] provides high-frequency, locally accurate short-term poses and visual loop closure [7] suppresses long-term drift to yield temporally consistent camera trajectories. On top of this backbone, open-vocabulary vision-language models [8]–[11] lift RGB-D observations into a voxelized 3D spatio-semantic map that is updated online as the scene changes, while a Redundancy-Aware Memory Pruning (RMP) scheme discards stale or redundant voxels and keeps memory growth bounded over long horizons. Building on this representation, DREAM implements an efficient hybrid localization and task-oriented navigation stack: a hybrid localizer combines language-conditioned feature-field retrieval, open-vocabulary detection, and multimodal LLM-based verification to recover reliable 3D target poses, and a value-guided exploration and navigation module operates on a dynamic semantic and obstacle map to prioritize uncertain or task-relevant regions and to select manipulation-aware docking poses near the target. Once the robot arrives near the target, DREAM leverages grasping primitives, geometry-based heuristics, and multi-round retry strategies to achieve robust pickup and placement in cluttered scenes. Our contributions are as follows:

- We introduce DREAM, a framework for mobile manipulation in dynamic, previously unseen indoor environments without any pre-built map.
- We design an online spatio-semantic memory that uses open-vocabulary vision-language models to build and update a voxelized 3D semantic map, and we propose an RMP scheme with pose-graph-aware reintegration and keyframe-guided pruning to bound memory growth and maintain efficient map updates.
- We develop a lightweight indoor LiDAR-Inertial-Visual multi-sensor SLAM backend that combines LiDAR-Inertial odometry with visual loop closure to deliver high-rate, low-drift, temporally consistent camera poses in dynamic indoor environments.
- We implement and evaluate DREAM on a real mobile manipulation robot in four dynamic indoor laboratory environments, demonstrating higher navigation, pickup, placement, and long-horizon task success, while using substantially less spatio-semantic memory and computation than prior voxel-based methods.

II. RELATED WORK

A. Open-Vocabulary Mobile Manipulation

Recent research on mobile manipulation has increasingly focused on open-vocabulary settings [3], enabling robots to execute user-specified tasks in everyday household environments.

In this setting, tasks are typically given as a single natural-language instruction, such as pick A from B and place C. Large language models (LLMs) [12], [13] and VLMs are employed to interpret the instruction and, via task planning [14], decompose it into a sequence of subtasks that are executed in order. Unlike pure navigation [15], mobile manipulation requires the robot to move into close proximity to target objects and operate in cluttered surroundings, so as to create favorable conditions for subsequent manipulation. Several works have explored tackling mobile manipulation using deep reinforcement learning [16], [17], but their success rates in real-world scenarios remain limited due to sim-to-real gaps and insufficient generalization. Other approaches utilize imitation learning to realize whole-body control to solve mobile manipulation problems [18], [19]; however, such methods typically succeed only on objects or environments encountered during training, lacking explicit scene memory and understanding and thus exhibiting very limited generalization. In contrast, modular frameworks [3]–[5] that combine language understanding, environment mapping, task planning, and low-level control to achieve open-vocabulary mobile manipulation have become the dominant choice in practice, as they tend to be more stable and better behaved on real robotic systems. Our system also follows this modular paradigm, with dynamic adaptation and active exploration.

B. 3D Semantic Scene Representation

Mainstream semantic 3D scene representations can be broadly divided into two categories: 3D voxel maps [3], [20] and 3D scene graphs [1], [5], [21]. A 3D voxel map assigns a semantic label to each voxel or point, thereby providing a certain degree of object localization capability. However, due to its dense grid structure, it tends to require substantial memory in large-scale environments and thus usually has to operate at a relatively low spatial resolution. In addition, this representation struggles to explicitly encode spatial relations between objects, which makes it less suitable for downstream planning modules that require complex spatial reasoning. In contrast, 3D scene graphs model objects as nodes and spatial or semantic relations as edges, explicitly capturing scene structure in a graphical form. This makes them particularly attractive for hierarchical navigation and large-scale navigation tasks. Nonetheless, when it comes to fusing geometry and semantics, constructing and maintaining a consistent 3D scene graph is computationally expensive. In tasks that require real-time scene reconstruction and continuous detection of environmental changes, their efficiency is typically much lower than that of 3D voxel maps. Recent work has leveraged NeRF [22], [23] or 3D Gaussian Splatting [24], [25] for scene reconstruction. While these methods can produce high-fidelity representations, real-time constraints generally force the assumption of a static environment. Extending them to dynamic scenes usually demands nearly prohibitive computational and time overheads, which makes them difficult to deploy for online mobile manipulation. For real-time and memory-efficient performance in dynamic environments, DREAM adopts a 3D voxel map as a pragmatic scene representation, together with a RMP mechanism that

selectively removes redundant or stale observations to contain memory growth.

C. Mapping and Navigation in Dynamic Environments

For navigation, Simultaneous Localization and Mapping (SLAM) is fundamental, as it determines the quality of geometric and semantic reconstruction and thus impacts downstream planning and manipulation. Some recent works evaluate purely in simulation under known or near-perfect robot state [26], [27]; others discard explicit maps and rely only on images [28], using large models to output end-to-end actions or short-horizon policies [29]. Although these approaches are promising in certain regimes, the lack of an explicit, maintainable scene memory makes it difficult to guarantee consistent behavior over long horizons in complex real-world environments. DynaMem [4] brings voxel-based semantic mapping to real-world mobile manipulation by using a simple SLAM system to provide camera poses and a 3D voxel map to detect scene changes and trigger memory updates, which in turn supports replanning in downstream navigation. However, its geometric backend relies mainly on short-term pose estimates and lacks strong loop closure and global optimization, effectively assuming that accumulated drift remains small. In more complex environments, drift and strong dynamics can violate this assumption, causing the internal memory to deviate from the true scene and leading to failures. In contrast, DREAM employs a lightweight LiDARInertialVisual SLAM backend with visual loop closure and pose-graph optimization to provide temporally consistent camera trajectories under dynamics, and maintains a dynamic spatio-semantic memory that stays geometrically registered to the environment, enabling reliable target (re)localization and long-horizon navigation/manipulation in dynamic environments.

III. METHOD

In this section, we present the overall architecture of DREAM and its three main modules: (i) a multi-sensor SLAM backend with a dynamic spatio-semantic voxel memory and redundancy-aware pruning, (ii) a hybrid localization and task-oriented navigation module, and (iii) a grasping and placement stack for mobile manipulation in dynamic, previously unseen indoor environments without any pre-built map.

A. Multi-Sensor SLAM and Dynamic Spatio-Semantic Memory

To support long-horizon mobile manipulation in dynamic, previously unseen environments, DREAM relies on a lightweight yet robust geometric backend and a continuously maintained 3D spatio-semantic memory. In this subsection, we first describe the LiDAR-Inertial-Visual SLAM backend that provides high-rate, low-drift camera poses and registered point clouds for downstream perception. We then introduce how DREAM constructs and updates a voxel-based spatio-semantic memory on top of this backbone, followed by a Redundancy-Aware Memory Pruning (RMP) mechanism that keeps the memory compact while preserving long-horizon consistency.

1) **Lightweight LiDARInertialVisual SLAM Backend:** We use an RGB-D camera and a 3D LiDAR with a built-in IMU to provide geometric and visual information for mapping and localization (see the top-left part of Fig. 2). Instead of performing a dedicated handeye calibration and hardware-level time synchronization, we obtain the extrinsic transforms from the robot CAD model and rely on software timestamp alignment, which simplifies deployment in real-world setups while remaining sufficient for our purposes thanks to downstream visual refinement.

For multi-sensor fusion, we first feed the LiDAR and internal IMU measurements from the MID-360 into Fast-LIO2 [6] to obtain a high-rate LiDAR-inertial odometry and de-skewed registered point clouds. These odometry estimates, together with RGB and depth images from the D435i, are then provided to RTAB-Map, which performs visual loop closure and global pose graph optimization to produce temporally consistent camera trajectories. RTAB-Map organizes incoming frames into keyframes and intermediate frames: only keyframes are inserted into the pose graph for long-term optimization, while intermediate frames are used locally and then discarded. This keyframe-based strategy substantially reduces the memory footprint required to store historical frames and prevents the SLAM backend from exhausting system resources during long-horizon operation in large indoor environments. Building on this SLAM backbone, we further develop a dynamic spatio-semantic memory that maintains low memory usage while preserving long-horizon consistency of the reconstructed scene.

2) **Dynamic Spatio-Semantic Memory:** We construct a continuously updatable spatio-semantic memory that maintains geometric and open-vocabulary semantic information of the environment. A sparse 3D voxel map is used as the underlying representation, and each non-empty voxel alongside its 3D location (x, y, z) , observation count C (how many times the voxel has been observed), the source image ID I (which frame the voxel was back-projected from), a high-dimensional semantic feature vector f extracted from a VLM such as CLIP [8] or SigLIP [30], and the latest observation time t in seconds, following the representation design of DynaMem [4]. To make this data structure dynamic, we describe the process of adding and updating new observations, as well as removing outdated objects and their associated voxels.

Adding Points. When DREAM receives a new set of observations from the output of RTAB-Map, i.e., RGB-D images, camera parameters, and the pose graph, we convert them into 3D coordinates in the global reference frame and generate a semantic feature vector for each point. Using the known camera transformation matrices, we compute global coordinates from the global camera poses and the back-projected depth images. We use pixel-level semantic features from SigLIP [30] and associate them with each 3D point. For more details on the mapping from images to feature vectors, we refer to prior work [3], [5]. Once we have computed the points and their associated features, we cluster the new points and assign them to the nearest voxel grid cells. The top-right of Fig. 2 illustrates how we update the metadata of each voxel. The count tracks the total number of points assigned to each voxel, and the feature vector tracks the weighted average of all feature vectors

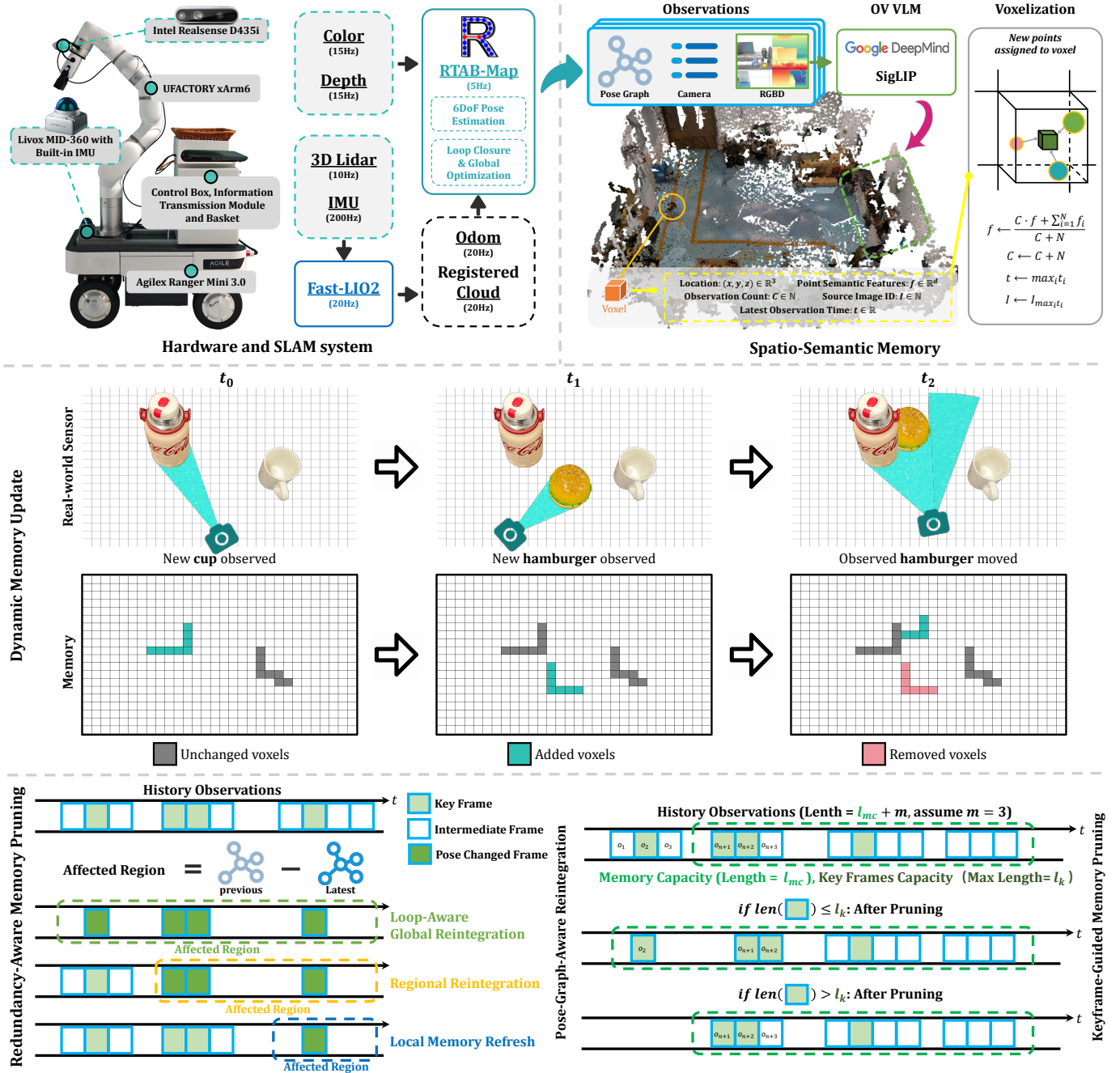


Fig. 2. **Hardware system and dynamic spatio-semantic memory in DREAM.** The multi-sensor SLAM backend provides temporally consistent poses to build a voxel-based spatio-semantic memory, which is continuously updated with incoming observations. A Redundancy-Aware Memory Pruning mechanism performs pose-graph-aware re-integration and keyframe-guided pruning to maintain a compact and consistent map in dynamic environments.

assigned to that voxel. Finally, we update the observation timestamp and image ID to keep track of the most recent observations that contributed to a given voxel. If a voxel is empty before assignment, we assume its count $C = 0$ and its feature vector f is the zero vector.

Removing Points. When an object is moved or removed in real-world environment, its associated voxels in DREAM memory may need to be removed as well. We use ray casting to identify obsolete voxels. This operation follows a simple principle: if a voxel lies inside the frustum between the camera plane and the corresponding view point cloud, then that voxel must be empty. To reduce the impact of long-range depth noise

and ensure that the map built from the current frame supports short-range navigation, we ignore pixels whose associated depth values exceed 3 m. A simplified 2D illustration of this algorithm is shown in the middle of Fig. 2. In practice, to accelerate intersection tests between the sparse voxel map and the viewing frustum, we project each existing voxel onto the camera plane and compute its distance to the camera. Suppose the image has height and width (H, W) , and the depth image is D . If a voxel is projected to image coordinates (h, w) on the camera plane with depth d , then the voxel is removed when d satisfies the following two conditions.

$$(h, w) \in [0, H] \times [0, W] \quad (1)$$

$$d \in (0, \min(3, \mathbf{D}[h, w] + \epsilon)) \quad (2)$$

Eq. 1 guarantees that the projected pixel falls inside the camera view, whereas Eq. 2 enforces that (a) $d > 0$ so the voxel is in front of the camera, (b) $d < 3\text{m}$ so it is not excessively far away, and (c) $d < \mathbf{D}[h, w]$ so it lies between the camera and the surface observed at pixel (h, w) .

3) Redundancy-Aware Memory Pruning: DREAM maintains a spatio-semantic voxel memory that grows over time as new observations are integrated. To keep the system scalable and robust to loop closures, we introduce a Redundancy-Aware Memory Pruning (RMP) mechanism that selectively updates the map according to pose-graph changes and prunes redundant observations. We treat all observations that appear as nodes in the pose graph as keyframes and denote the keyframe set by $\mathcal{K}$. For each keyframe, we compare the camera pose used during its original integration with the optimized pose from the pose graph. Let $\Delta \mathbf{t}$ and $\Delta \theta$ denote the translation and rotation differences; if $\|\Delta \mathbf{t}\|_2 > \tau_{\text{trans}}$ or $\Delta \theta > \tau_{\text{rot}}$, where τ_{trans} and τ_{rot} are translational and rotational thresholds, the keyframe is marked as pose-inconsistent and added to $\mathcal{K}_u \subset \mathcal{K}$.

Pose-Graph-Aware Reintegration. We perform pose-graph-aware reintegration based on how many recent keyframes are pose-inconsistent. Let w_{rt} and w_{sm} be the lengths of the real-time and short-term windows over the keyframe history, and let $\mathcal{K}_{\text{rt}}$ and $\mathcal{K}_{\text{sm}}$ be the corresponding subsets of $\mathcal{K}$. When a large fraction of keyframes is pose-inconsistent, i.e., $|\mathcal{K}_u|/|\mathcal{K}| > \rho_L$ with $\rho_L = 2/3$, we regard this as a large loop closure and reset the voxel-level semantic memory, followed by reintegrating all keyframes in chronological order using their optimized poses (*Loop-Aware Global Reintegration*, see the bottom-left of Fig. 2). If instead more than a fraction $\rho_M \approx 1/3$ of the keyframes in the short-term window are pose-inconsistent, i.e., $|\mathcal{K}_u \cap \mathcal{K}_{\text{sm}}|/|\mathcal{K}_{\text{sm}}| > \rho_M$, and at least one pose-inconsistent keyframe lies in the real-time window ($|\mathcal{K}_u \cap \mathcal{K}_{\text{rt}}| > 0$), we reintegrate only keyframes in the last w_{sm} observations (*Regional Reintegration*). Otherwise, if $|\mathcal{K}_u \cap \mathcal{K}_{\text{rt}}| > 0$ but the above two conditions are not met, we reintegrate keyframes only within $\mathcal{K}_{\text{rt}}$ to perform a lightweight local refresh (*Local Memory Refresh*); otherwise, the pose update is ignored for the spatio-semantic memory.

Keyframe-Guided Memory Pruning. Let l_{mc} denote the memory capacity (maximum history length), m the overflow length, and l_k the maximum number of keyframes allowed in memory, as illustrated in the bottom-right of Fig. 2. When the history length reaches $l_{\text{mc}} + m$, we remove the m oldest observations in a keyframe-aware manner: intermediate frames are discarded first, and keyframes are removed only when their count exceeds l_k . To further reduce the cost of storing and decoding high-dimensional semantic features, we retain feature vectors only for keyframes and for the most recent w_{rt} observations, which serve as a buffer for real-time updates, while older non-keyframe observations keep only geometric information. In combination, pose-graph-aware reintegration and keyframe-guided pruning keep the spatio-semantic memory compact and consistent over long horizons without introducing significant additional computation.

B. Hybrid Localization and Task-Oriented Navigation

In this subsection, we detail how DREAM leverages the dynamic spatio-semantic memory constructed in previous sections to support hybrid localization and task-oriented navigation, and how it reacquires targets after they have been temporarily lost.

1) Hybrid Localization: Our hybrid localizer builds on VLMs such as CLIP [8] and SigLIP [30], which embed images and text into a shared latent space. This enables DREAM to perform coarse semantic retrieval directly in the spatio-semantic memory, followed by image-space detection and high-level semantic verification. As described in Sec. III-A, incoming RGB-D observations are first converted into per-point visual features and integrated into a voxelized spatio-semantic memory. Given a language query, we encode the text using the VLM text encoder and compute its latent embedding. We then perform a coarse localization step by identifying the voxel whose stored feature exhibits the highest inner product with this text embedding. Conditioned on the selected voxel, we retrieve the most recent associated RGB image from the spatio-semantic memory and apply an open-vocabulary object detector (e.g., OWL-v2 [11]) to detect instances of the queried category in image space. The 3D position of the candidate object is then obtained by back-projecting the center pixel of the detected bounding box into the world coordinate frame.

A purely similarity-threshold-based approach is, however, brittle in realistic scenes that contain multiple semantically or visually similar objects. For instance, when the language query is “hot dog”, but the explored scene only contains a “hamburger”, a method that selects the highest-scoring match in latent space may still confidently return the hamburger and its 3D location as the answer to the “hot dog” query. In contrast, multimodal large language models (mLLMs) [12] are considerably better at resolving such fine-grained semantic distinctions. We therefore introduce an mLLM-based semantic verification stage on top of the retrieval and detection pipeline. Concretely, our hybrid localization pipeline consists of three stages, illustrated in Fig. 3: (i) spatio-semantic-memorybased coarse localization, which retrieves the highest-confidence voxel together with its most recent associated image; (ii) open-vocabulary detection on this image to generate 3D object hypotheses for the queried category; and (iii) mLLM-based re-ranking and verification, which filters out detections whose visual evidence is inconsistent with the language query. This multi-stage design substantially improves robustness, allowing DREAM to localize the *actual* target object present in the explored scene rather than the closest semantic surrogate.

If no candidate survives the verification stage, we treat the query as not grounded in the current memory, mark the object as “object not found”, and hand control to the exploration policy described next. Meanwhile, RMP prunes the spatio-semantic memory by discarding stale observations, so retrieval and verification operate over a bounded set of voxels and images, keeping the hybrid localizer efficient even in long-horizon tasks.

2) Task-Oriented Navigation and Exploration: To realize task-oriented navigation in real-world environments, DREAM

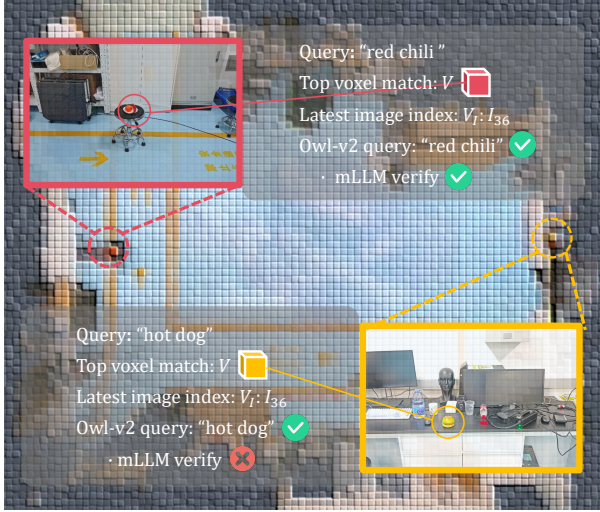


Fig. 3. **Hybrid Localization.** The VLM feature field in the spatio-semantic memory is first queried with a language embedding to retrieve a high-confidence voxel and its associated RGB frame, which is then passed through an open-vocabulary detector to obtain object proposals. Finally, an mLLM inspects the retrieved image and either confirms or rejects the presence of the queried object, yielding a verified 3D target pose.

combines a classical grid-based A* [31] planner with the dynamic spatio-semantic memory. Depending on whether the target object is currently localized, the system either tracks a known target pose or enters an exploration mode in which it actively searches for the object using a value-based exploration map and distance-aware heuristics. When the robot approaches the object, we further introduce a notion of *docking point* to obtain base poses that are favorable for manipulation, and execute paths in a receding-horizon fashion to remain robust in dynamic environments.

Obstacle Map Construction. We first construct a 2D obstacle map from the spatio-semantic memory. A sparse 3D point cloud is normalized with respect to the sensor height and the ground plane is translated to $z = 0$. To account for noise in depth measurements, we introduce a ground threshold (0.2 m in our experiments): voxels above this height are marked as obstacles, while voxels below are projected onto a 2D grid as navigable cells. Grid cells that are neither labeled as obstacles nor as navigable are treated as explorable frontier cells, on which the planner can expand the explored region.

Exploration and Goal Selection. The navigation module takes as input the robot’s current pose and a target pose. When the hybrid localization module in Sec. III-B1 is confident about the target pose, the system treats it as a navigation goal and runs A* on the obstacle map to compute a shortest path between the start and target, as illustrated in Fig. 4(a). If, instead, Hybrid Localization returns “object not found”, DREAM switches to an exploration mode in which the planner selects frontier cells as intermediate goals. Rather than exploring uniformly, we bias this frontier-based exploration using a *value-based exploration map*, shown in Fig. 4(b), to quickly steer the robot toward promising regions. This map is decomposed into two terms. The first term is a *recency map* (Fig. 4(c)) that encourages the robot to revisit areas that have not been observed for a long time. It prioritizes locations that are “least recently observed”. Let the current time be T and

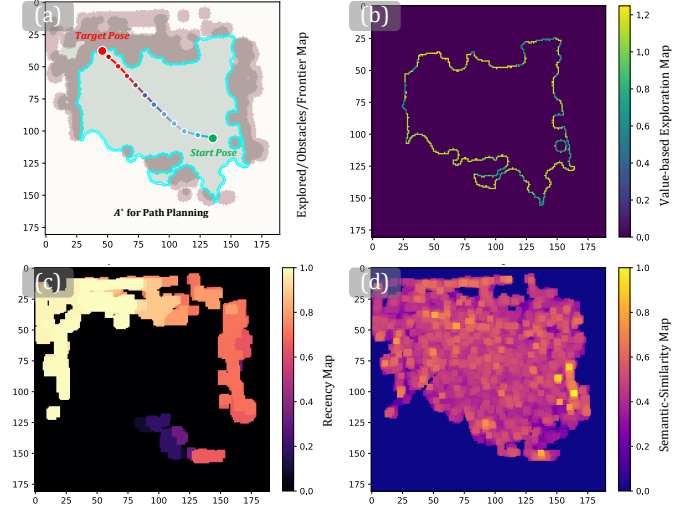


Fig. 4. **Task-oriented Navigation and Exploration.** (a) A* path planning on the explored/obstacle/frontier map from the current start pose to the task-specific target pose. (b) Exploration score map obtained by combining the recency and semantic-similarity values; high-score frontier cells are selected as exploration goals. (c) Recency map V_T that assigns a higher value to regions that have not been observed for a long time. (d) Semantic-similarity map V_S computed from VLM features, highlighting regions whose appearance is most relevant to the language query.

the last-seen time of voxel (x, y, z) be $t_{x,y,z}$. The recency map V_T is defined as

$$T^*[x, y] = \max_z (T - t_{x,y,z}), \quad (3)$$

$$V_T[x, y] = \sigma(-\beta_T (T^*[x, y] - \mu_T)), \quad (4)$$

where β_T and μ_T are hyperparameters, and σ denotes the sigmoid function. The second term is a *semantic-similarity map* (Fig. 4(d)) that exploits the VLM features stored in the spatio-semantic memory. Let $f_{x,y,z}$ be the VLM feature at voxel (x, y, z) and f_q be the VLM feature of the language query. The semantic-similarity map V_S is defined as

$$S^*[x, y] = \max_z (f_q \cdot f_{x,y,z}), \quad (5)$$

$$V_S[x, y] = \sigma(-\beta_S (S^*[x, y] - \mu_S)), \quad (6)$$

where β_S and μ_S are also hyperparameters. A linear combination of V_T and V_S defines the exploration score map in Fig. 4(b), balancing temporal novelty and semantic relevance.

During exploration, we further impose a distance-dependent heuristic to reduce local oscillations when sampling exploration goals from this score map. Let $p_t \in \mathbb{R}^2$ denote the robot base position at time t , and let $\mathcal{G}_t$ be the set of frontier cells sorted by their exploration score. We first identify frontier cells that lie outside a small neighborhood of the base,

$$\mathcal{F}_t = \{g \in \mathcal{G}_t \mid \|g - p_t\|_2 \geq d_e\}, \quad (7)$$

with $d_e = 0.85$ m in our implementation. In practice, the planner samples the next exploration goal from

$$\mathcal{G}_t^{\text{explore}} = \begin{cases} \mathcal{F}_t, & \text{if } \mathcal{F}_t \neq \emptyset, \\ \mathcal{G}_t, & \text{otherwise,} \end{cases} \quad (8)$$

i.e., it prefers high-score frontier cells at least d_e away from the current base when such cells exist, and falls back to all frontier

cells when the remaining unexplored area is confined to a small neighborhood around the robot. This behavior prevents the planner from repeatedly selecting goals directly under or immediately next to the base, which would lead to meaningless in-place exploration.

Docking and Receding-horizon Execution. Once the target object has been localized, navigation near the goal is driven by manipulation requirements rather than pure coverage. Let $o_t \in \mathbb{R}^2$ denote the 2D position of the queried object at time t . We define a manipulation range d_m , also set to 0.85 m, and introduce the notion of a *docking point*: a base pose that places the robot within reach of the object while keeping a collision-free corridor between the base and the object. Formally, let $\mathcal{D}_t$ be a set of navigable cells in a neighborhood of o_t ; we consider feasible docking candidates in

$$\mathcal{D}_t^{\text{dock}} = \{g \in \mathcal{D}_t \mid \|g - o_t\|_2 \leq d_m, \text{localFree}(g, o_t)\}, \quad (9)$$

where $\text{localFree}(g, o_t)$ denotes a lightweight local collision check that rejects base goals whose direct corridor to the object is occupied in the obstacle map. In our implementation, the docking point g_t^{dock} is chosen from $\mathcal{D}_t^{\text{dock}}$ as the goal with the lowest path cost from p_t . This encourages the robot to stop at configurations that leave sufficient clearance and line of sight for manipulation, instead of halting too far from or directly behind nearby obstacles.

To handle dynamic environments, we execute navigation in a semi-closed-loop, receding-horizon fashion. Instead of executing the full A* path in one shot, the robot follows only the first 16 waypoints (about 1-1.5 m of motion), then stops to re-sense the environment, update the map, and replan. This allows the planner to react to moving obstacles and scene changes. Once the robot has effectively reached a docking point, i.e., $\|p_t - o_t\|_2 \leq d_m$, we disable this truncation to avoid unnecessary detours near the goal. The robot repeats this cycle until its distance to the target falls below a predefined threshold and no obstacles block manipulation; at that point, the mobile base is aligned to face the object and the camera pose is adjusted so that the target is projected near the center of the image plane.

C. Grasping and Placement Strategy

Manipulation is a central component of DREAM and, in turn, imposes strong requirements on navigation: the base must approach the target with sufficient visibility, reachability, and collision-free workspace for the arm. Once the robot has reached a suitable docking point and the target object is visible to the wrist-mounted RGB-D camera, DREAM triggers a grasping or placement routine, as illustrated in Fig. 5.

1) *Grasping Strategy*: Despite recent progress in Vision Language Action (VLA) Models [32], [33], their robustness and open-vocabulary grasping capability in real-world settings remain limited. We therefore adopt a modular grasping stack with three components: (i) grasp proposal generation with AnyGrasp, (ii) a heuristic for top-down grasps, and (iii) a two-stage execution policy.

AnyGrasp. Given a language-specified target, once the robot has navigated to a docking point (Sec. III-B), the wrist-mounted RGB-D camera is oriented toward the 3D position of

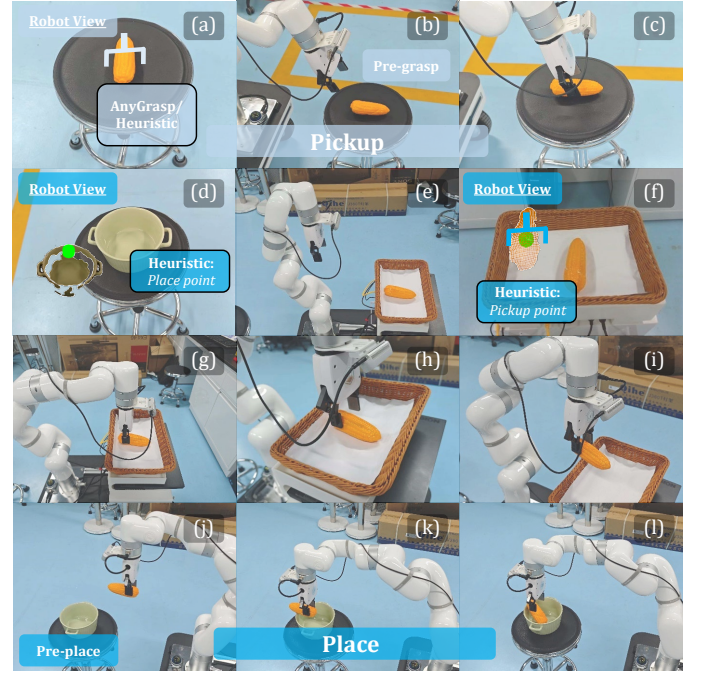


Fig. 5. **Grasping and placement in DREAM.** (a)(c) AnyGrasp / heuristic grasp proposal, pre-grasp pose, and executed pickup. (d)(f) Heuristic selection of the placement point in the bowl and the pickup point in the basket. (g)(i) Two-stage approach and grasp execution inside the basket. (j)(l) Pre-place pose, descent, and final placement of the object in the bowl.

the object and an RGB-D image of the local scene is captured. The RGB image is processed by an open-vocabulary detector and segmentor (Grounding DINO [9] + Segment Anything-2 [10]) to obtain a pixel-level mask of the queried object. Using camera intrinsics and extrinsics, the RGB-D observation is back-projected into a 3D point cloud in the robot base frame and passed to AnyGrasp [34] to generate collision-free grasp candidates. We further filter these candidates using the segmentation mask, retaining only those whose contact regions lie entirely on the segmented object, and execute the remaining candidate with the highest confidence. This language- and segmentation-conditioned filtering yields grasp poses that are specific to the queried object rather than nearby distractors.

Heuristic grasp. In real-world experiments, AnyGrasp does not always return valid grasps, especially on specular or noisy surfaces where depth is unreliable. We therefore add a fallback heuristic that is invoked only when AnyGrasp fails. It assumes a top-down grasp and sets the gripper yaw according to the shape of the segmented object point cloud. We project the segmented object into the horizontal plane and subtract the median (x, y) to obtain centered points $p_i \in \mathbb{R}^2$. From these points we compute the covariance matrix and take its two orthogonal eigenvectors e_1, e_2 (principal axis and its complement). For each axis e_k we measure the extent

$$\ell_k = \max_i(p_i \cdot e_k) - \min_i(p_i \cdot e_k), \quad k \in \{1, 2\}. \quad (10)$$

The gripper closing direction is set to the shorter axis,

$$c = \begin{cases} e_1, & \ell_1 \leq \ell_2, \\ e_2, & \text{otherwise,} \end{cases} \quad \theta_{\text{yaw}} = \text{atan2}(c_y, c_x), \quad (11)$$

where (c_x, c_y) are the components of c and θ_{yaw} is wrapped to $(-\pi, \pi)$. For elongated objects this aligns the closing direction with the shorter axis of the footprint; for nearly isotropic shapes either axis is acceptable. In practice, this provides a simple, shape-adaptive yaw estimate whenever learned grasps are unavailable.

Two-stage execution. Once a grasp pose has been selected, we execute it using a short, staged approach to ensure safe and stable contact. Let $\vec{p}$ denote the grasp point in the base frame and $\vec{a}$ a unit approach direction (for top-down grasps aligned with the world z -axis). The end-effector follows the trajectory

$$\langle \vec{p} - 0.25 \vec{a}, \vec{p} \rangle, \quad (12)$$

moving along $\vec{a}$ from a pre-grasp waypoint to the final grasp point; see Fig. 5(b),(c). This keeps the arm in a collision-free configuration until the last phase of the motion and reduces the risk of disturbing the object during approach. The gripper is then closed in a simple feedback-controlled manner: starting from an initial opening, we iteratively reduce the gripper width by 5 mm and monitor the resulting motion. When a closure command produces only a small additional movement while the gripper has not reached its mechanical limit, we regard the object as firmly grasped. The robot then lifts the object and places it into a storage basket mounted at the rear of the base, completing the pickup phase.

2) *Placement Strategy:* The placement phase starts from the configuration where the object has been successfully transported to and stored in the rear basket, and the robot is commanded to move it to a new target region (e.g., a bowl), as illustrated in Fig. 5(d). The procedure comprises two stages: (i) estimating a safe placement pose inside the target container, and (ii) re-grasping the object from the basket (Fig. 5(e)-(i)) and executing a two-stage placement motion (Fig. 5(j)-(l)).

The robot first navigates to a docking point facing the target container and reuses the same open-vocabulary perception stack as in grasping. Grounding DINO and Segment Anything 2 are applied to the RGB image to obtain a segmentation mask of the container, and the masked pixels are back-projected with depth to form a point cloud $\mathcal{P}^{\text{place}} = \{(x, y, z)\}$ in the robot base frame. The median horizontal coordinates (x_m, y_m) provide a robust estimate of the region center. Around this center, we consider points within a small window ($|x - x_m| < 0.1$ m, $|y - y_m| < 0.1$ m) and determine the maximum support height inside this window. The safe placement height is defined as

$$z_{\text{max}} = 0.1 + \max\{z \mid |x - x_m| < 0.1, |y - y_m| < 0.1\}, \quad (13)$$

where the constant 0.1 m is a vertical safety margin above the local support surface. This yields a top-down placement pose at $(x_m, y_m, z_{\text{max}})$, shown in Fig. 5(d). After fixing the placement pose, the arm is redirected toward the rear basket, and we reuse the same heuristic top-down grasping procedure as in the pickup phase to grasp the object from the basket, as illustrated in Fig. 5(f). Finally, with the object firmly held, the arm moves from the basket to the pre-computed placement pose using the same two-stage pattern as in grasping: it first goes to a pre-place waypoint above $(x_m, y_m, z_{\text{max}})$, then follows a straight-line trajectory to the final pose before opening the gripper

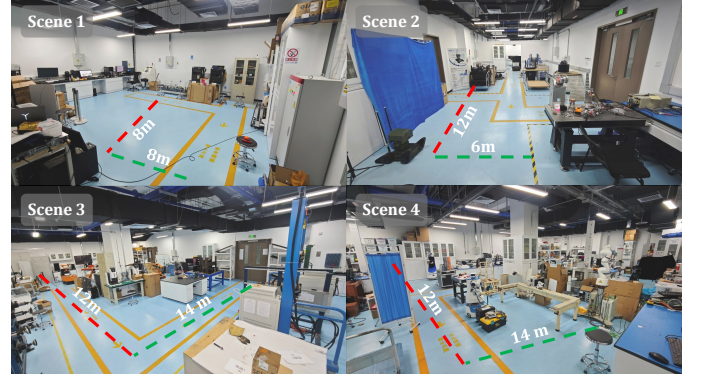


Fig. 6. **Real-world laboratory scenes used for evaluation.** S1-S4 are dynamic indoor rooms of size 8×8 , 6×12 , and 12×14 m (for both S3 and S4), with obstacle density and corridor narrowness increasing from S1 to S4.

(Fig. 5(j)-(l)), which minimizes lateral motion and reduces the likelihood of collisions with the container or surrounding environment.

IV. EXPERIMENTS

In this section, we evaluate DREAM on mobile manipulation tasks using a real-world mobile manipulation platform in four dynamic indoor laboratory scenes. We focus on three aspects: target navigation and exploration, object pickup and placement, and spatio-semantic memory efficiency. We first describe the experimental setups for two core task categories: (i) mobile manipulation in dynamic environments, including the success rate and robustness of the navigation, pickup, and placement stages; and (ii) memory consumption and update efficiency of the scene representation during long-term operation. Finally, we present ablation studies to analyze the contribution of each key module to overall performance.

A. Hardware Platform and Real-world Environments

We evaluate DREAM on the mobile manipulation platform shown in the top-left of Fig. 2. The platform consists of an AgileX Ranger Mini 3.0 mobile base, a UFACTORY xArm6 manipulator, an Intel RealSense D435i RGB-D camera mounted on the wrist, and a Livox MID-360 LiDAR with a built-in IMU. All algorithms, including our method and the baselines, run on the same on-board computing platform and share the same low-level motion control stack of the mobile base as well as the same SLAM-based localization module. We evaluate the system in four real-world laboratory scenes (S1-S4; 64 m^2 , 72 m^2 , 168 m^2 , 168 m^2) as shown in Fig. 6, where obstacle density and corridor narrowness increase progressively from S1 to S4. In each scene, multiple everyday household objects are randomly placed as potential manipulation targets. During task execution, we deliberately change the position of the target object after the robot has initially localized it, in order to emulate human intervention and dynamic scene changes in realistic environments. This allows us to assess the capability of DREAM to maintain a consistent scene representation and to adapt its behavior in highly dynamic settings.

TABLE I
SUCCESS RATES (%) ON DYNAMIC MOBILE MANIPULATION TASKS. WE
COMPARE DREAM WITH DYNAMEM ACROSS FOUR REAL-WORLD SCENES.

Task	Method	S1	S2	S3	S4
Navigation	DynaMem	85.7 (30/35)	82.4 (28/34)	72.7 (24/33)	68.8 (22/32)
	DREAM	89.2 (33/37)	86.1 (31/36)	83.8 (31/37)	85.2 (23/27)
Pickup	DynaMem	88.2 (15/17)	87.5 (14/16)	92.9 (13/14)	92.3 (12/13)
	DREAM	94.4 (17/18)	94.1 (16/17)	94.4 (17/18)	93.8 (15/16)
Place	DynaMem	92.3 (12/13)	91.7 (11/12)	80.0 (8/10)	88.9 (8/9)
	DREAM	93.3 (14/15)	92.9 (13/14)	92.3 (12/13)	91.7 (11/12)
Long-term	DynaMem	60.0 (12/20)	55.0 (11/20)	40.0 (8/20)	40.0 (8/20)
	DREAM	70.0 (14/20)	65.0 (13/20)	60.0 (12/20)	55.0 (11/20)

B. Mobile Manipulation in Dynamic Environments

In this task, given a natural-language instruction (e.g., “put the orange into the green bowl”), the robot must operate in an unknown and dynamically changing environment to complete the full pipeline from task-oriented navigation and exploration to pickup and placement. We define the following four evaluation metrics: **Navigation Success Rate**: The robot must actively explore and navigate from the initial pose to the vicinity of the target. We regard navigation as successful if, when it stops, the Euclidean distance to the target is less than 1 m and the target is fully visible in the wrist camera image, even when the target has been moved one or more times by a human during execution. **Pickup Success Rate**: Conditioned on successful navigation, a trial is counted as a success if the robot stably grasps the target and transports it into the rear storage basket. **Place Success Rate**: Conditioned on successful navigation, a trial is counted as a success if the robot places the target stably inside or on the specified container. **Long-term Success Rate**: Each trial consists of Navigation, Pickup, a second Navigation, and Place. Long-term success requires all four stages to succeed.

Results. The quantitative results are summarized in Tab. I. Across all four scenes, DREAM consistently outperforms DynaMem on Navigation, Pickup, Place, and Long-term success rates. On the navigation stage, DREAM improves the success rate from 68.8%-85.7% (DynaMem) to 83.8%-89.2% across the four scenes. The gain is most pronounced in the more cluttered scenes (S3 and S4), where the environment contains many visually and semantically similar distractor objects. This suggests that the proposed hybrid localization can more reliably lock onto the true target object and recover from temporary target relocation. For manipulation, DREAM also achieves consistently higher Pickup and Place success rates. Pickup success improves from 87.5%-92.9% for DynaMem to 93.8%-94.4% for DREAM, while Place success increases from 80.0%-92.3% to 91.7%-93.3%. We attribute these gains to the combination of AnyGrasp with the heuristic fallback and the two-stage execution strategy, which together make grasping and placement more robust under partial occlusion and depth noise. When considering the full task pipeline, DREAM achieves Long-term success rates of 55.0%-70.0%, compared to 40.0%-60.0% for DynaMem, corresponding to improvements of 10-20 percentage points in all four scenes. Importantly, these gains are obtained under frequent human interventions that relocate the target object during execution,

showing that DREAM can reacquire moved targets and complete long-horizon manipulation tasks in dynamic, previously unseen environments. Overall, these results indicate that the proposed integration of dynamic spatio-semantic memory, hybrid localization, and task-oriented navigation improves the robustness of mobile manipulation in dynamic real-world environments.

C. Memory Consumption and Update Efficiency

To fairly compare the memory consumption and update efficiency of different scene representations under identical conditions, we adopt a log-and-replay protocol. Specifically, in each scene we first teleoperate the robot along a predefined exploration trajectory, driving it through the room while using a unified SLAM system to record a continuous stream of RGB-D observations together with the corresponding camera/base poses. During this logging phase, we deliberately move some of the target objects to introduce scene dynamics caused by human intervention. This phase is used purely for data collection and does not build any scene representation. We then feed the same RGB-D and pose sequence to DovSG [5], DynaMem [4], and DREAM respectively. Starting from an empty map, each method incrementally reconstructs and updates its own scene representation frame by frame. For each scene, we record the final memory footprint of the representation and the average per-frame update time over the entire sequence. In all four scenes, we use a spatial resolution of 0.05 m for all methods.

Results. Tab. II reports the memory footprint and average per-frame update time for DovSG, DynaMem, and DREAM across all four scenes. As the number of processed frames increases, the voxel count and memory usage of DynaMem grow rapidly, reaching up to 10.43 GB in S4. Its per-frame update time also increases to more than 2 s in the larger scenes. DovSG maintains a more compact memory footprint (below 1 GB in all scenes) thanks to its node-level semantic representation, but its per-frame update time exceeds 110 s, dominated by repeated calls to heavy vision-language models and graph maintenance. This makes it impractical for real-time online operation on our robotic platform. In contrast, DREAM keeps both memory and update time nearly flat as the task progresses. Across all four scenes, DREAM uses only 0.37-0.63 GB of memory, which is roughly 3-15 $\times$ smaller than DynaMem at the same map resolution, and achieves average per-frame update times of 0.43-0.53 s, which are about 2-4 $\times$ faster than DynaMem, even as loop closures and dynamic scene changes accumulate over long trajectories. These results show that the proposed Redundancy-Aware Memory Pruning allows DREAM to maintain an accurate long-horizon spatio-semantic representation while keeping memory and computation bounded during long-term operation in dynamic environments.

D. Ablation Studies

To analyze the contribution of the key components in DREAM to the overall performance, we conduct ablation studies on the grasping module and the hybrid localization module.

TABLE II

MEMORY USAGE AND PER-FRAME UPDATE TIME ACROSS SCENES.
MEMORY IS MEASURED IN GB, TIME IN SECONDS (S). LOWER IS BETTER.

Method	Memory (GB)↓				Time (s)↓			
	S1	S2	S3	S4	S1	S2	S3	S4
DovSG	0.23	0.35	0.83	0.89	113	115	124	128
DynaMem	1.32	2.45	8.96	10.43	0.82	1.11	2.21	2.38
DREAM	0.37	0.46	0.61	0.63	0.43	0.49	0.53	0.52

TABLE III

ABLATION STUDY OF PICKUP AND HYBRID LOCALIZATION MODULES.
SUCCESS RATES (%) AVERAGED OVER ALL TRIALS.

Task	Method / Variant	Success ↑
Pickup	Heuristic only	67
	AnyGrasp only	79
	AnyGrasp + Heuristic (no two-stage)	88
	AnyGrasp + Heuristic + two-stage (DREAM)	92
Query object	VLM (CLIP) + OWL-V2	75
	VLM (SigLIP) + OWL-V2	77
	Full hybrid localization (DREAM)	83

(1) Grasping module. We compare the following configurations: (i) *Full configuration*: AnyGrasp for grasp candidate generation + heuristic for grasp pose generation + two-stage trajectory execution; (ii) *Without two-stage execution*: AnyGrasp for grasp candidate generation and the heuristic for grasp pose generation are retained, but only a single-stage grasp trajectory is executed; (iii) *AnyGrasp only*: directly execute the grasp candidates output by AnyGrasp without heuristic or two-stage execution; (iv) *Heuristic only*: do not use AnyGrasp; grasp poses are generated purely by the heuristic and executed directly. These variants are evaluated in a tabletop setting with a fixed-base manipulator. We use ten common household objects to test the grasping-related modules. During evaluation, each object is placed in different poses and is attempted ten times, resulting in 100 grasp trials per configuration.

Results. Tab. III (top) summarizes the Pickup success rates of different grasping configurations in a tabletop setting. Using only the heuristic top-down grasp yields a 67% success rate, while using only AnyGrasp improves this to 79%. Combining AnyGrasp with the heuristic refinement (without two-stage execution) further increases the success rate to 88%, and the full configuration with two-stage execution reaches 92%. These results indicate that (i) learned grasp proposals are important for difficult objects, (ii) the heuristic refinement helps when AnyGrasp fails or returns suboptimal poses, and (iii) the two-stage execution reduces collisions and stabilizes contact, leading to the best overall performance.

(2) Hybrid localization module. For the hybrid localization mechanism, we evaluate the following variants: (i) *VLM feature field + OWL-V2 (SigLIP)*: use a VLM feature field built from SigLIP features combined with OWL-V2 detection, and determine the target position via nearest-neighbor retrieval in the feature space; (ii) *VLM feature field + OWL-V2 (CLIP)*: replace SigLIP with CLIP features while keeping the rest of the pipeline unchanged; (iii) *Full hybrid localization*: build on VLM feature retrieval and OWL-V2 detection, and additionally introduce an mLLM-based semantic verification step to filter and confirm candidate targets. We adopt the same log-and-

replay protocol as in the memory experiment to collect RGB-D and pose sequences. Before data collection, we arrange objects in each scene to satisfy two conditions: (i) the scene contains several object categories that are truly present and will be used as query targets; and (ii) we deliberately add a number of semantically or visually similar distractor objects (e.g., peppers of different colors from the same category, or a hamburger placed near a hot dog) to assess the robustness of each variant in the presence of distractors. We then follow the DREAM pipeline to build a scene representation from the recorded RGB-D and pose sequence. Once the scene is fully reconstructed, we select 20 object categories that actually exist in the scene as query targets. For each variant, we run 100 query episodes in total across the four scenes.

Results. Tab. III (bottom) reports the success rates for localizing queried objects in the reconstructed scenes. Using a CLIP-based feature field with OWL-V2 detection achieves a 75% success rate. Replacing CLIP with SigLIP provides a modest improvement to 77%, suggesting slightly better open-vocabulary retrieval. The full hybrid localization of DREAM, which adds an mLLM-based semantic verification stage on top of the SigLIP feature field and OWL-V2 detections, achieves an 83% success rate. This shows that semantic verification is effective at rejecting spurious nearest-neighbor matches and is particularly beneficial when the scene contains multiple visually similar distractors or when the queried category is absent. Together with the feature-field retrieval and open-vocabulary detection described in Sec. III-B1 and Fig. 3, this confirms that the full hybrid localization pipeline is necessary to robustly ground language queries in complex, dynamic scenes.

V. CONCLUSIONS

In this work, we introduced DREAM, a mobile manipulation framework for dynamic, previously unseen indoor environments without any pre-built map. By coupling a lightweight LiDARInertialVisual SLAM backend with a dynamic spatio-semantic voxel memory and a Redundancy-Aware Memory Pruning mechanism, DREAM maintains a compact yet temporally consistent 3D representation under frequent scene changes. On top of this representation, we build a hybrid localization module, a value-guided exploration and navigation scheme, and a robust graspingplacement stack that jointly enable language-conditioned target acquisition, reacquisition, and manipulation in cluttered real-world settings. Extensive experiments on a real mobile manipulation platform across four dynamic indoor laboratory scenes demonstrate that DREAM significantly improves Navigation, Pickup, Place, and Long-term success rates over a strong voxel-map baseline, and achieves a favorable balance between memory usage and computational efficiency.

REFERENCES

- [1] Q. Gu, A. Kuwajerwala, S. Morin, and et al., “Conceptgraphs: Open-vocabulary 3d scene graphs for perception and planning,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.16650>

- [2] A. Werby, C. Huang, M. Büchner, and et al., “Hierarchical open-vocabulary 3d scene graphs for language-grounded robot navigation,” in *Robotics: Science and Systems XX*, ser. RSS2024. Robotics: Science and Systems Foundation, Jul. 2024. [Online]. Available: <http://dx.doi.org/10.15607/RSS.2024.XX.077>
- [3] P. Liu, Y. Orru, C. Paxton, and et al., “Ok-robot: What really matters in integrating open-knowledge models for robotics,” *arXiv preprint arXiv:2401.12202*, 2024.
- [4] P. Liu, Z. Guo, M. Warke, and et al., “Dynamem: Online dynamic spatio-semantic memory for open world mobile manipulation,” 2024.
- [5] Z. Yan, S. Li, Z. Wang, and et al., “Dynamic open-vocabulary 3d scene graphs for long-term language-guided mobile manipulation,” 2025. [Online]. Available: <https://arxiv.org/abs/2410.11989>
- [6] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, “Fast-lio2: Fast direct lidar-inertial odometry,” 2021. [Online]. Available: <https://arxiv.org/abs/2107.06829>
- [7] M. Labbé and F. Michaud, “Rtabmap as an opensource lidar and visual simultaneous localization and mapping library for largescale and longterm online operation,” *Journal of Field Robotics*, vol. 36, no. 2, p. 416446, Oct. 2018. [Online]. Available: <http://dx.doi.org/10.1002/rob.21831>
- [8] A. Radford, J. W. Kim, C. Hallacy, and et al., “Learning transferable visual models from natural language supervision,” 2021. [Online]. Available: <https://arxiv.org/abs/2103.00020>
- [9] S. Liu, Z. Zeng, T. Ren, and et al., “Grounding dino: Marrying dino with grounded pre-training for open-set object detection,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.05499>
- [10] N. Ravi, V. Gabeur, Y.-T. Hu, and et al., “Sam 2: Segment anything in images and videos,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.00714>
- [11] M. Minderer, A. Gritsenko, and N. Houlsby, “Scaling open-vocabulary object detection,” 2024. [Online]. Available: <https://arxiv.org/abs/2306.09683>
- [12] OpenAI and et al., “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [13] G. T. et al., “Gemini: A family of highly capable multimodal models,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.11805>
- [14] B. Li and et al., “Interactive task planning with language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2310.10645>
- [15] A. Bar, G. Zhou, D. Tran, and et al., “Navigation world models,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.03572>
- [16] A. Shukla, S. Tao, and H. Su, “Maniskill-hab: A benchmark for low-level manipulation in home rearrangement tasks,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.13211>
- [17] J. Li, X. Cheng, T. Huang, and et al., “Amo: Adaptive motion optimization for hyper-dexterous humanoid whole-body control,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.03738>
- [18] Z. Fu, T. Z. Zhao, and C. Finn, “Mobile aloha: Learning bimanual mobile manipulation with low-cost whole-body teleoperation,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.02117>
- [19] Q. Liao, T. E. Truong, X. Huang, and et al., “Beyondmimic: From motion tracking to versatile humanoid control via guided diffusion,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.08241>
- [20] N. (Mahi)Shafuallah, C. Paxton, L. Pinto, and et al., “Clip-fields: Weakly supervised semantic fields for robotic memory,” in *Robotics: Science and Systems XIX*, ser. RSS2023. Robotics: Science and Systems Foundation, Jul. 2023. [Online]. Available: <http://dx.doi.org/10.15607/RSS.2023.XIX.074>
- [21] H. Jiang, B. Huang, R. Wu, and et al., “Roboexp: Action-conditioned scene graph via interactive exploration for robotic manipulation,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.15487>
- [22] A. Rosinol, J. J. Leonard, and L. Carlone, “Nerf-slam: Real-time dense monocular slam with neural radiance fields,” 2022. [Online]. Available: <https://arxiv.org/abs/2210.13641>
- [23] D. Maggio, M. Abate, J. Shi, and et al., “Loc-nerf: Monte carlo localization using neural radiance fields,” 2022. [Online]. Available: <https://arxiv.org/abs/2209.09050>
- [24] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” 2023. [Online]. Available: <https://arxiv.org/abs/2308.04079>
- [25] L. Ge, X. Zhu, Z. Yang, and X. Li, “Dynamicgsg: Dynamic 3d gaussian scene graphs for environment adaptation,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.15309>
- [26] Z. Wu, Z. Wang, X. Xu, and et al., “Embodied instruction following in unknown environments,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.11818>
- [27] H. Yin, X. Xu, L. Zhao, and et al., “Unigoal: Towards universal zero-shot goal-oriented navigation,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.10630>
- [28] J. Haviland, N. Sunderhauf, and P. Corke, “A holistic approach to reactive mobile manipulation,” *IEEE Robotics and Automation Letters*, vol. 7, no. 2, p. 31223129, Apr. 2022. [Online]. Available: <http://dx.doi.org/10.1109/LRA.2022.3146554>
- [29] A.-C. Cheng, Y. Ji, Z. Yang, and et al., “Navila: Legged robot vision-language-action model for navigation,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.04453>
- [30] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.15343>
- [31] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [32] Y. Wang, X. Li, W. Wang, and et al., “Unified vision-language-action model,” 2025. [Online]. Available: <https://arxiv.org/abs/2506.19850>
- [33] Y. Liao, P. Zhou, S. Huang, and et al., “Genie envisioner: A unified world foundation platform for robotic manipulation,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.05635>
- [34] H.-S. Fang, C. Wang, and H. F. and et al., “Anygrasp: Robust and efficient grasp perception in spatial and temporal domains,” 2023. [Online]. Available: <https://arxiv.org/abs/2212.08333>

Zhijie Yan This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version. This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version. This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version.

Shufei Li This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version. This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version. This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version.

Ze Zhang This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version. This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version.

Xin Liu This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version. This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version.



Yuhang Zheng This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version. This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version.



Zuoxu Wang This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version. This is a placeholder biography used only to estimate the final length of the manuscript. The text will be replaced with the author's actual biography in the camera-ready version.